

Table of Contents

1. Intro To Dp
2. Knapsack
3. Longest_increasing_subsequence
4. Divide And Conquer Dp
5. Knuth Optimization
6. Profile Dynamics
7. Zero_matrix

Original

Introduction to Dynamic Programming

The essence of dynamic programming is to avoid repeated calculation. Often, dynamic programming problems are naturally solvable by recursion. In such cases, it's easiest to write the recursive solution, then save repeated states in a lookup table. This process is known as top-down dynamic programming with memoization. That's read "memoization" (like we are writing in a memo pad) not memorization.

One of the most basic, classic examples of this process is the fibonacci sequence. Its recursive formulation is $f(n) = f(n - 1) + f(n - 2)$ where $n \geq 2$ and $f(0) = 0$ and $f(1) = 1$. In C++, this would be expressed as:

```
int f(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    return f(n - 1) + f(n - 2);
}
```

The runtime of this recursive function is exponential - approximately $O(2^n)$ since one function call ($f(n)$) results in 2 similarly sized function calls ($f(n - 1)$ and $f(n - 2)$).

Speeding up Fibonacci with Dynamic Programming (Memoization)

Our recursive function currently solves fibonacci in exponential time. This means that we can only handle small input values before the problem becomes too difficult. For instance, $f(29)$ results in *over 1 million* function calls!

To increase the speed, we recognize that the number of subproblems is only $O(n)$. That is, in order to calculate $f(n)$ we only need to know $f(n - 1)$, $f(n - 2)$, ..., $f(0)$. Therefore, instead of recalculating these subproblems, we solve them once and then save the result in a lookup table. Subsequent calls will use this lookup table and immediately return a result, thus eliminating exponential work!

Each recursive call will check against a lookup table to see if the value has been calculated. This is done in $O(1)$ time. If we have previously calculated it, return the result, otherwise, we calculate the function normally. The overall runtime is $O(n)$. This is an enormous improvement over our previous exponential time algorithm!

```
const int MAXN = 100;
bool found[MAXN];
int memo[MAXN];

int f(int n) {
    if (found[n]) return memo[n];
    if (n == 0) return 0;
    if (n == 1) return 1;

    found[n] = true;
    return memo[n] = f(n - 1) + f(n - 2);
}
```

With our new memoized recursive function, $f(29)$, which used to result in *over 1 million calls*, now results in *only 57 calls*, nearly *20,000 times* fewer function calls! Ironically, we are now limited by our data type. $f(46)$ is the last fibonacci number that can fit into a signed 32-bit integer.

Typically, we try to save states in arrays, if possible, since the lookup time is $O(1)$ with minimal overhead. However, more generically, we can save states any way we like. Other examples include binary search trees (`map` in C++) or hash tables (`unordered_map` in C++).

An example of this might be:

```
unordered_map<int, int> memo;
int f(int n) {
    if (memo.count(n)) return memo[n];
    if (n == 0) return 0;
    if (n == 1) return 1;
```

```
    return memo[n] = f(n - 1) + f(n - 2);
}
```

Or analogously:

```
map<int, int> memo;
int f(int n) {
    if (memo.count(n)) return memo[n];
    if (n == 0) return 0;
    if (n == 1) return 1;

    return memo[n] = f(n - 1) + f(n - 2);
}
```

Both of these will almost always be slower than the array-based version for a generic memoized recursive function. These alternative ways of saving state are primarily useful when saving vectors or strings as part of the state space.

The layman's way of analyzing the runtime of a memoized recursive function is:

work per subproblem * number of subproblems

Using a binary search tree (map in C++) to save states will technically result in $O(n \log n)$ as each lookup and insertion will take $O(\log n)$ work and with $O(n)$ unique subproblems we have $O(n \log n)$ time.

This approach is called top-down, as we can call the function with a query value and the calculation starts going from the top (queried value) down to the bottom (base cases of the recursion), and makes shortcuts via memoization on the way.

Bottom-up Dynamic Programming

Until now you've only seen top-down dynamic programming with memoization. However, we can also solve problems with bottom-up dynamic programming. Bottom-up is exactly the opposite of top-down, you start at the bottom (base cases of the recursion), and extend it to more and more values.

To create a bottom-up approach for fibonacci numbers, we initialize the base cases in an array. Then, we simply use the recursive definition on array:

```
const int MAXN = 100;
int fib[MAXN];

int f(int n) {
    fib[0] = 0;
    fib[1] = 1;
    for (int i = 2; i <= n; i++) fib[i] = fib[i - 1] + fib[i - 2];

    return fib[n];
}
```

Of course, as written, this is a bit silly for two reasons: Firstly, we do repeated work if we call the function more than once. Secondly, we only need to use the two previous values to calculate the current element. Therefore, we can reduce our memory from $O(n)$ to $O(1)$.

An example of a bottom-up dynamic programming solution for fibonacci which uses $O(1)$ memory might be:

```
const int MAX_SAVE = 3;
int fib[MAX_SAVE];

int f(int n) {
    fib[0] = 0;
    fib[1] = 1;
    for (int i = 2; i <= n; i++)
        fib[i % MAX_SAVE] = fib[(i - 1) % MAX_SAVE] + fib[(i - 2) % MAX_SAVE];

    return fib[n % MAX_SAVE];
}
```

Note that we've changed the constant from `MAXN` to `MAX_SAVE`. This is because the total number of elements we need to access is only 3. It no longer scales with the size of input and is, by definition, $O(1)$ memory. Additionally, we use a common trick (using the modulo operator) only maintaining the values we need.

That's it. That's the basics of dynamic programming: Don't repeat the work you've done before.

One of the tricks to getting better at dynamic programming is to study some of the classic examples.

Classic Dynamic Programming Problems

Name	Description/Example
0-1 Knapsack	Given W , N , and N items with weights w_i and values v_i , what is the maximum $\sum_{i=1}^k v_i$ for each subset of items of size k ($1 \leq k \leq N$) while ensuring $\sum_{i=1}^k w_i \leq W$?
Subset Sum	Given N integers and T , determine whether there exists a subset of the given set whose elements sum up to the T .
Longest Increasing Subsequence (LIS)	You are given an array containing N integers. Your task is to determine the LIS in the array, i.e., a subsequence where every element is larger than the previous one.
Counting Paths in a 2D Array	Given N and M , count all possible distinct paths from $(1, 1)$ to (N, M) , where each step is either from (i, j) to $(i + 1, j)$ or $(i, j + 1)$.
Longest Common Subsequence	You are given strings s and t . Find the length of the longest string that is a subsequence of both s and t .
Longest Path in a Directed Acyclic Graph (DAG)	Finding the longest path in Directed Acyclic Graph (DAG).
Longest Palindromic Subsequence	Finding the Longest Palindromic Subsequence (LPS) of a given string.
Rod Cutting	Given a rod of length n units, Given an integer array $cuts$ where $cuts[i]$ denotes a position you should perform a cut at. The cost of one cut is the length of the rod to be cut. What is the minimum total cost of the cuts.
Edit Distance	The edit distance between two strings is the minimum number of operations required to transform one string into the other. Operations are ["Add", "Remove", "Replace"]

Related Topics

- [Bitmask Dynamic Programming](#)
- [Digit Dynamic Programming](#)
- [Dynamic Programming on Trees](#)

Of course, the most important trick is to practice.

Practice Problems

- [LeetCode - 1137. N-th Tribonacci Number](#)
- [LeetCode - 118. Pascal's Triangle](#)
- [LeetCode - 1025. Divisor Game](#)
- [Codeforces - Vacations](#)
- [Codeforces - Hard problem](#)
- [Codeforces - Zuma](#)
- [LeetCode - 221. Maximal Square](#)
- [LeetCode - 1039. Minimum Score Triangulation of Polygon](#)

DP Contests

- [Atcoder - Educational DP Contest](#)
- [CSES - Dynamic Programming](#)

Contributors:

[Michael Hayter](#) (80.0%) [Zyad M. Ayad](#) (9.09%) [Alexander Zhipa](#) (3.64%) [Konstantinos Alatzas](#) (1.82%) [Shubham Phapale](#) (1.82%) [Ahmed Marouf](#) (1.21%)
[Jakob Kogler](#) (1.21%) [Oleksandr Kulkov](#) (0.61%) [Tejwinder Singh](#) (0.61%)

Original

Knapsack Problem

Prerequisite knowledge: [Introduction to Dynamic Programming](#)

Introduction

Consider the following example:

[USACO07 Dec] Charm Bracelet

There are n distinct items and a knapsack of capacity W . Each item has 2 attributes, weight (w_i) and value (v_i). You have to select a subset of items to put into the knapsack such that the total weight does not exceed the capacity W and the total value is maximized.

In the example above, each object has only two possible states (taken or not taken), corresponding to binary 0 and 1. Thus, this type of problem is called "0-1 knapsack problem".

0-1 Knapsack

Explanation

In the example above, the input to the problem is the following: the weight of i^{th} item w_i , the value of i^{th} item v_i , and the total capacity of the knapsack W .

Let $f_{i,j}$ be the dynamic programming state holding the maximum total value the knapsack can carry with capacity j , when only the first i items are considered.

Assuming that all states of the first $i - 1$ items have been processed, what are the options for the i^{th} item?

- When it is not put into the knapsack, the remaining capacity remains unchanged and total value does not change. Therefore, the maximum value in this case is $f_{i-1,j}$
- When it is put into the knapsack, the remaining capacity decreases by w_i and the total value increases by v_i , so the maximum value in this case is $f_{i-1,j-w_i} + v_i$

From this we can derive the dp transition equation:

$$f_{i,j} = \max(f_{i-1,j}, f_{i-1,j-w_i} + v_i)$$

Further, as f_i is only dependent on f_{i-1} , we can remove the first dimension. We obtain the transition rule

$$f_j \leftarrow \max(f_j, f_{j-w_i} + v_i)$$

that should be executed in the **decreasing** order of j (so that f_{j-w_i} implicitly corresponds to $f_{i-1,j-w_i}$ and not $f_{i,j-w_i}$).

It is important to understand this transition rule, because most of the transitions for knapsack problems are derived in a similar way.

Implementation

The algorithm described can be implemented in $O(nW)$ as:

```
for (int i = 1; i <= n; i++)
    for (int j = W; j >= w[i]; j--)
        f[j] = max(f[j], f[j - w[i]] + v[i]);
```

Again, note the order of execution. It should be strictly followed to ensure the following invariant: Right before the pair (i, j) is processed, f_k corresponds to $f_{i,k}$ for $k > j$, but to $f_{i-1,k}$ for $k < j$. This ensures that f_{j-w_i} is taken from the $(i-1)$ -th step, rather than from the i -th one.

Complete Knapsack

The complete knapsack model is similar to the 0-1 knapsack, the only difference from the 0-1 knapsack is that an item can be selected an unlimited number of times instead of only once.

We can refer to the idea of 0-1 knapsack to define the state: $f_{i,j}$, the maximum value the knapsack can obtain using the first i items with maximum capacity j .

It should be noted that although the state definition is similar to that of a 0-1 knapsack, its transition rule is different from that of a 0-1 knapsack.

Explanation

The trivial approach is, for the first i items, enumerate how many times each item is to be taken. The time complexity of this is $O(n^2W)$.

This yields the following transition equation:

$$f_{i,j} = \max_{k=0}^{\infty} (f_{i-1,j-k \cdot w_i} + k \cdot v_i)$$

At the same time, it simplifies into a "flat" equation:

$$f_{i,j} = \max(f_{i-1,j}, f_{i,j-w_i} + v_i)$$

The reason this works is that $f_{i,j-w_i}$ has already been updated by $f_{i,j-2 \cdot w_i}$ and so on.

Similar to the 0-1 knapsack, we can remove the first dimension to optimize the space complexity. This gives us the same transition rule as 0-1 knapsack.

$$f_j \leftarrow \max(f_j, f_{j-w_i} + v_i)$$

Implementation

The algorithm described can be implemented in $O(nW)$ as:

```
for (int i = 1; i <= n; i++)
    for (int j = w[i]; j <= W; j++)
        f[j] = max(f[j], f[j - w[i]] + v[i]);
```

Despite having the same transition rule, the code above is incorrect for 0-1 knapsack.

Observing the code carefully, we see that for the currently processed item i and the current state $f_{i,j}$, when $j \geq w_i$, $f_{i,j}$ will be affected by $f_{i,j-w_i}$. This is equivalent to being able to put item i into the backpack multiple times, which is consistent with the complete knapsack problem and not the 0-1 knapsack problem.

Multiple Knapsack

Multiple knapsack is also a variant of 0-1 knapsack. The main difference is that there are k_i of each item instead of just 1.

Explanation

A very simple idea is: "choose each item k_i times" is equivalent to " k_i of the same item is selected one by one". Thus converting it to a 0-1 knapsack model, which can be described by the transition function:

$$f_{i,j} = \max_{k=0}^{k_i} (f_{i-1,j-k \cdot w_i} + k \cdot v_i)$$

The time complexity of this process is $O(W \sum_{i=1}^n k_i)$

Binary Grouping Optimization

We still consider converting the multiple knapsack model into a 0-1 knapsack model for optimization. The time complexity $O(Wn)$ can not be further optimized with the approach above, so we focus on $O(\sum k_i)$ component.

Let $A_{i,j}$ denote the j^{th} item split from the i^{th} item. In the trivial approach discussed above, $A_{i,j}$ represents the same item for all $j \leq k_i$. The main reason for our low efficiency is that we are doing a lot of repetitive work. For example, consider selecting $\{A_{i,1}, A_{i,2}\}$, and selecting $\{A_{i,2}, A_{i,3}\}$. These two situations are completely equivalent. Thus optimizing the splitting method will greatly reduce the time complexity.

The grouping is made more efficient by using binary grouping.

Specifically, $A_{i,j}$ holds 2^j individual items ($j \in [0, \lfloor \log_2(k_i + 1) \rfloor - 1]$). If $k_i + 1$ is not an integer power of 2, another bundle of size $k_i - (2^{\lfloor \log_2(k_i + 1) \rfloor} - 1)$ is used to make up for it.

Through the above splitting method, it is possible to obtain any sum of $\leq k_i$ items by selecting a few $A_{i,j}$'s. After splitting each item in the described way, it is sufficient to use 0-1 knapsack method to solve the new formulation of the problem.

This optimization gives us a time complexity of $O(W \sum_{i=1}^n \log k_i)$.

Implementation

```
index = 0;
for (int i = 1; i <= n; i++) {
    int c = 1, p, h, k;
    cin >> p >> h >> k;
    while (k > c) {
        k -= c;
        list[++index].w = c * p;
        list[index].v = c * h;
        c *= 2;
    }
    list[++index].w = p * k;
    list[index].v = h * k;
}
```

Monotone Queue Optimization

In this optimization, we aim to convert the knapsack problem into a [maximum queue](#) one.

For convenience of description, let $g_{x,y} = f_{i,x \cdot w_i + y}$, $g'_{x,y} = f_{i-1,x \cdot w_i + y}$. Then the transition rule can be written as:

$$g_{x,y} = \max_{k=0}^{k_i} (g'_{x-k,y} + v_i \cdot k)$$

Further, let $G_{x,y} = g'_{x,y} - v_i \cdot x$. Then the transition rule can be expressed as:

$$g_{x,y} \leftarrow \max_{k=0}^{k_i} (G_{x-k,y}) + v_i \cdot x$$

This transforms into a classic monotone queue optimization form. $G_{x,y}$ can be calculated in $O(1)$, so for a fixed y , we can calculate $g_{x,y}$ in $O(\lfloor \frac{W}{w_i} \rfloor)$ time. Therefore, the complexity of finding all $g_{x,y}$ is $O(\lfloor \frac{W}{w_i} \rfloor) \times O(w_i) = O(W)$. In this way, the total complexity of the algorithm is reduced to $O(nW)$.

Mixed Knapsack

The mixed knapsack problem involves a combination of the three problems described above. That is, some items can only be taken once, some can be taken infinitely, and some can be taken at most k times.

The problem may seem daunting, but as long as you understand the core ideas of the previous knapsack problems and combine them together, you can do it. The pseudo code for the solution is as:

```
for (each item) {  
    if (0-1 knapsack)  
        Apply 0-1 knapsack code;  
    else if (complete knapsack)  
        Apply complete knapsack code;  
    else if (multiple knapsack)  
        Apply multiple knapsack code;  
}
```

Practise Problems

- [Atcoder: Knapsack-1](#)
- [Atcoder: Knapsack-2](#)
- [LeetCode - 494. Target Sum](#)
- [LeetCode - 416. Partition Equal Subset Sum](#)
- [CSES: Book Shop II](#)
- [DMOJ: Knapsack-3](#)
- [DMOJ: Knapsack-4](#)

Contributors:

[Akshat Nagpal](#) (97.83%) [Ahmed Marouf](#) (1.09%) [Oleksandr Kulkov](#) (0.54%) [virinci](#) (0.54%)

Translated

From: e-maxx.ru

Longest increasing subsequence

We are given an array with n numbers: $a[0 \dots n - 1]$. The task is to find the longest, strictly increasing, subsequence in a .

Formally we look for the longest sequence of indices $i_1, \dots, i_k$ such that

$$i_1 < i_2 < \dots < i_k, \quad a[i_1] < a[i_2] < \dots < a[i_k]$$

In this article we discuss multiple algorithms for solving this task. Also we will discuss some other problems, that can be reduced to this problem.

Solution in $O(n^2)$ with dynamic programming

Dynamic programming is a very general technique that allows to solve a huge class of problems. Here we apply the technique for our specific task.

First we will search only for the **length** of the longest increasing subsequence, and only later learn how to restore the subsequence itself.

Finding the length

To accomplish this task, we define an array $d[0 \dots n - 1]$, where $d[i]$ is the length of the longest increasing subsequence that ends in the element at index i .

Example

$$a = \{8, 3, 4, 6, 5, 2, 0, 7, 9, 1\}$$

$$d = \{1, 1, 2, 3, 3, 1, 1, 4, 5, 2\}$$

The longest increasing subsequence that ends at index 4 is $\{3, 4, 5\}$ with a length of 3, the longest ending at index 8 is either $\{3, 4, 5, 7, 9\}$ or $\{3, 4, 6, 7, 9\}$, both having length 5, and the longest ending at index 9 is $\{0, 1\}$ having length 2.

We will compute this array gradually: first $d[0]$, then $d[1]$, and so on. After this array is computed, the answer to the problem will be the maximum value in the array $d[]$.

So let the current index be i . I.e. we want to compute the value $d[i]$ and all previous values $d[0], \dots, d[i - 1]$ are already known. Then there are two options:

- $d[i] = 1$: the required subsequence consists only of the element $a[i]$.
- $d[i] > 1$: The subsequence will end at $a[i]$, and right before it will be some number $a[j]$ with $j < i$ and $a[j] < a[i]$.

It's easy to see, that the subsequence ending in $a[j]$ will itself be one of the longest increasing subsequences that ends in $a[j]$. The number $a[i]$ just extends that longest increasing subsequence by one number.

Therefore, we can just iterate over all $j < i$ with $a[j] < a[i]$, and take the longest sequence that we get by appending $a[i]$ to the longest increasing subsequence ending in $a[j]$. The longest increasing subsequence ending in $a[j]$ has length $d[j]$, extending it by one gives the length $d[j] + 1$.

$$d[i] = \max_{\substack{j < i \\ a[j] < a[i]}} (d[j] + 1)$$

If we combine these two cases we get the final answer for $d[i]$:

$$d[i] = \max \left(1, \max_{\substack{j < i \\ a[j] < a[i]}} (d[j] + 1) \right)$$

Implementation

Here is an implementation of the algorithm described above, which computes the length of the longest increasing subsequence.

```
int lis(vector<int> const& a) {
    int n = a.size();
    vector<int> d(n, 1);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (a[j] < a[i])
                d[i] = max(d[i], d[j] + 1);
        }
    }

    int ans = d[0];
    for (int i = 1; i < n; i++) {
        ans = max(ans, d[i]);
    }
    return ans;
}
```

Restoring the subsequence

So far we only learned how to find the length of the subsequence, but not how to find the subsequence itself.

To be able to restore the subsequence we generate an additional auxiliary array $p[0 \dots n - 1]$ that we will compute alongside the array $d[]$. $p[i]$ will be the index j of the second last element in the longest increasing subsequence ending in i . In other words the index $p[i]$ is the same index j at which the highest value $d[i]$ was obtained. This auxiliary array $p[]$ points in some sense to the ancestors.

Then to derive the subsequence, we just start at the index i with the maximal $d[i]$, and follow the ancestors until we deduced the entire subsequence, i.e. until we reach the element with $d[i] = 1$.

Implementation of restoring

We will change the code from the previous sections a little bit. We will compute the array $p[]$ alongside $d[]$, and afterwards compute the subsequence.

For convenience we originally assign the ancestors with $p[i] = -1$. For elements with $d[i] = 1$, the ancestors value will remain -1 , which will be slightly more convenient for restoring the subsequence.

```
vector<int> lis(vector<int> const& a) {
    int n = a.size();
    vector<int> d(n, 1), p(n, -1);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (a[j] < a[i] && d[i] < d[j] + 1) {
                d[i] = d[j] + 1;
                p[i] = j;
            }
        }
    }

    int ans = d[0], pos = 0;
    for (int i = 1; i < n; i++) {
        if (d[i] > ans) {
            ans = d[i];
            pos = i;
        }
    }

    vector<int> subseq;
    while (pos != -1) {
        subseq.push_back(a[pos]);
    }
```

```

    pos = p[pos];
}
reverse(subseq.begin(), subseq.end());
return subseq;
}

```

Alternative way of restoring the subsequence

It is also possible to restore the subsequence without the auxiliary array $p[]$. We can simply recalculate the current value of $d[i]$ and also see how the maximum was reached.

This method leads to a slightly longer code, but in return we save some memory.

Solution in $O(n \log n)$ with dynamic programming and binary search

In order to obtain a faster solution for the problem, we construct a different dynamic programming solution that runs in $O(n^2)$, and then later improve it to $O(n \log n)$.

We will use the dynamic programming array $d[0 \dots n]$. This time $d[l]$ doesn't correspond to the element $a[i]$ or to a prefix of the array. $d[l]$ will be the smallest element at which an increasing subsequence of length l ends.

Initially we assume $d[0] = -\infty$ and for all other lengths $d[l] = \infty$.

We will again gradually process the numbers, first $a[0]$, then $a[1]$, etc, and in each step maintain the array $d[]$ so that it is up to date.

Example

Given the array $a = \{8, 3, 4, 6, 5, 2, 0, 7, 9, 1\}$, here are all their prefixes and their dynamic programming array. Notice, that the values of the array don't always change at the end.

prefix = {}	$d = \{-\infty, \infty, \dots\}$
prefix = {8}	$d = \{-\infty, 8, \infty, \dots\}$
prefix = {8, 3}	$d = \{-\infty, 3, \infty, \dots\}$
prefix = {8, 3, 4}	$d = \{-\infty, 3, 4, \infty, \dots\}$
prefix = {8, 3, 4, 6}	$d = \{-\infty, 3, 4, 6, \infty, \dots\}$
prefix = {8, 3, 4, 6, 5}	$d = \{-\infty, 3, 4, 5, \infty, \dots\}$
prefix = {8, 3, 4, 6, 5, 2}	$d = \{-\infty, 2, 4, 5, \infty, \dots\}$
prefix = {8, 3, 4, 6, 5, 2, 0}	$d = \{-\infty, 0, 4, 5, \infty, \dots\}$
prefix = {8, 3, 4, 6, 5, 2, 0, 7}	$d = \{-\infty, 0, 4, 5, 7, \infty, \dots\}$
prefix = {8, 3, 4, 6, 5, 2, 0, 7, 9}	$d = \{-\infty, 0, 4, 5, 7, 9, \infty, \dots\}$
prefix = {8, 3, 4, 6, 5, 2, 0, 7, 9, 1}	$d = \{-\infty, 0, 1, 5, 7, 9, \infty, \dots\}$

When we process $a[i]$, we can ask ourselves. What have the conditions to be, that we write the current number $a[i]$ into the $d[0 \dots n]$ array?

We set $d[l] = a[i]$, if there is a longest increasing sequence of length l that ends in $a[i]$, and there is no longest increasing sequence of length l that ends in a smaller number. Similar to the previous approach, if we remove the number $a[i]$ from the longest increasing sequence of length l , we get another longest increasing sequence of length $l - 1$. So we want to extend a longest increasing sequence of length $l - 1$ by the number $a[i]$, and obviously the longest increasing sequence of length $l - 1$ that ends with the smallest element will work the best, in other words the sequence of length $l - 1$ that ends in element $d[l - 1]$.

There is a longest increasing sequence of length $l - 1$ that we can extend with the number $a[i]$, exactly if $d[l - 1] < a[i]$. So we can just iterate over each length l , and check if we can extend a longest increasing sequence of length $l - 1$ by checking the criteria.

Additionally we also need to check, if we maybe have already found a longest increasing sequence of length l with a smaller number at the end. So we only update if $a[i] < d[l]$.

After processing all the elements of $a[]$ the length of the desired subsequence is the largest l with $d[l] < \infty$.

```
int lis(vector<int> const& a) {
    int n = a.size();
    const int INF = 1e9;
    vector<int> d(n+1, INF);
    d[0] = -INF;

    for (int i = 0; i < n; i++) {
        for (int l = 1; l <= n; l++) {
            if (d[l-1] < a[i] && a[i] < d[l])
                d[l] = a[i];
        }
    }

    int ans = 0;
    for (int l = 0; l <= n; l++) {
        if (d[l] < INF)
            ans = l;
    }
    return ans;
}
```

We now make two important observations.

1. The array d will always be sorted: $d[l-1] < d[l]$ for all $i = 1 \dots n$.

This is trivial, as you can just remove the last element from the increasing subsequence of length l , and you get a increasing subsequence of length $l-1$ with a smaller ending number.

2. The element $a[i]$ will only update at most one value $d[l]$.

This follows immediately from the above implementation. There can only be one place in the array with $d[l-1] < a[i] < d[l]$.

Thus we can find this element in the array $d[]$ using [binary search](#) in $O(\log n)$. In fact we can simply look in the array $d[]$ for the first number that is strictly greater than $a[i]$, and we try to update this element in the same way as the above implementation.

Implementation

This gives us the improved $O(n \log n)$ implementation:

```
int lis(vector<int> const& a) {
    int n = a.size();
    const int INF = 1e9;
    vector<int> d(n+1, INF);
    d[0] = -INF;

    for (int i = 0; i < n; i++) {
        int l = upper_bound(d.begin(), d.end(), a[i]) - d.begin();
        if (d[l-1] < a[i] && a[i] < d[l])
            d[l] = a[i];
    }

    int ans = 0;
    for (int l = 0; l <= n; l++) {
        if (d[l] < INF)
            ans = l;
    }
    return ans;
}
```

Restoring the subsequence

It is also possible to restore the subsequence using this approach. This time we have to maintain two auxiliary arrays. One that tells us the index of the elements in $d[]$. And again we have to create an array of "ancestors" $p[i]$. $p[i]$ will be the index of the previous element for the optimal subsequence ending in element i .

It's easy to maintain these two arrays in the course of iteration over the array $a[]$ alongside the computations of $d[]$. And at the end it is not difficult to restore the desired subsequence using these arrays.

Solution in $O(n \log n)$ with data structures

Instead of the above method for computing the longest increasing subsequence in $O(n \log n)$ we can also solve the problem in a different way: using some simple data structures.

Let's go back to the first method. Remember that $d[i]$ is the value $d[j] + 1$ with $j < i$ and $a[j] < a[i]$.

Thus if we define an additional array $t[]$ such that

$$t[a[i]] = d[i],$$

then the problem of computing the value $d[i]$ is equivalent to finding the **maximum value in a prefix** of the array $t[]$:

$$d[i] = \max(t[0 \dots a[i] - 1] + 1)$$

The problem of finding the maximum of a prefix of an array (which changes) is a standard problem that can be solved by many different data structures. For instance we can use a [Segment tree](#) or a [Fenwick tree](#).

This method has obviously some **shortcomings**: in terms of length and complexity of the implementation this approach will be worse than the method using binary search. In addition if the input numbers $a[i]$ are especially large, then we would have to use some tricks, like compressing the numbers (i.e. renumber them from 0 to $n - 1$), or use a dynamic segment tree (only generate the branches of the tree that are important). Otherwise the memory consumption will be too high.

On the other hand this method has also some **advantages**: with this method you don't have to think about any tricky properties in the dynamic programming solution. And this approach allows us to generalize the problem very easily (see below).

Related tasks

Here are several problems that are closely related to the problem of finding the longest increasing subsequence.

Longest non-decreasing subsequence

This is in fact nearly the same problem. Only now it is allowed to use identical numbers in the subsequence.

The solution is essentially also nearly the same. We just have to change the inequality signs, and make a slight modification to the binary search.

Number of longest increasing subsequences

We can use the first discussed method, either the $O(n^2)$ version or the version using data structures. We only have to additionally store in how many ways we can obtain longest increasing subsequences ending in the values $d[i]$.

The number of ways to form a longest increasing subsequences ending in $a[i]$ is the sum of all ways for all longest increasing subsequences ending in j where $d[j]$ is maximal. There can be multiple such j , so we need to sum all of them.

Using a Segment tree this approach can also be implemented in $O(n \log n)$.

It is not possible to use the binary search approach for this task.

Smallest number of non-increasing subsequences covering a sequence

For a given array with n numbers $a[0 \dots n - 1]$ we have to colorize the numbers in the smallest number of colors, so that each color forms a non-increasing subsequence.

To solve this, we notice that the minimum number of required colors is equal to the length of the longest increasing subsequence.

Proof: We need to prove the **duality** of these two problems.

Let's denote by x the length of the longest increasing subsequence and by y the least number of non-increasing subsequences that form a cover. We need to prove that $x = y$.

It is clear that $y < x$ is not possible, because if we have x strictly increasing elements, then no two can be part of the same non-increasing subsequence. Therefore we have $y \geq x$.

We now show that $y > x$ is not possible by contradiction. Suppose that $y > x$. Then we consider any optimal set of y non-increasing subsequences. We transform this in set in the following way: as long as there are two such subsequences such that the first begins before the second subsequence, and the first sequence start with a number greater than or equal to the second, then we unhook this starting number and attach it to the beginning of second. After a finite number of steps we have y subsequences, and their starting numbers will form an increasing subsequence of length y . Since we assumed that $y > x$ we reached a contradiction.

Thus it follows that $y = x$.

Restoring the sequences: The desired partition of the sequence into subsequences can be done greedily. I.e. go from left to right and assign the current number or that subsequence ending with the minimal number which is greater than or equal to the current one.

Practice Problems

- [ACMSGURU - "North-East"](#)
- [Codeforces - LCIS](#)
- [Codeforces - Tourist](#)
- [SPOJ - DOSA](#)
- [SPOJ - HMLIS](#)
- [SPOJ - ONEXLIS](#)
- [SPOJ - SUPPER](#)
- [Topcoder - AutoMarket](#)
- [Topcoder - BridgeArrangement](#)
- [Topcoder - IntegerSequence](#)
- [UVA - Back To Edit Distance](#)
- [UVA - Happy Birthday](#)
- [UVA - Tiling Up Blocks](#)

Contributors:

[Michael Hayter](#) (100.0%)

Original

Divide and Conquer DP

Divide and Conquer is a dynamic programming optimization.

Preconditions

Some dynamic programming problems have a recurrence of this form:

$$dp(i, j) = \min_{0 \leq k \leq j} dp(i-1, k-1) + C(k, j)$$

where $C(k, j)$ is a cost function and $dp(i, j) = 0$ when $j < 0$.

Say $0 \leq i < m$ and $0 \leq j < n$, and evaluating C takes $O(1)$ time. Then the straightforward evaluation of the above recurrence is $O(mn^2)$. There are $m \times n$ states, and n transitions for each state.

Let $opt(i, j)$ be the value of k that minimizes the above expression. Assuming that the cost function satisfies the quadrangle inequality, we can show that $opt(i, j) \leq opt(i, j+1)$ for all i, j . This is known as the *monotonicity condition*. Then, we can apply divide and conquer DP. The optimal "splitting point" for a fixed i increases as j increases.

This lets us solve for all states more efficiently. Say we compute $opt(i, j)$ for some fixed i and j . Then for any $j' < j$ we know that $opt(i, j') \leq opt(i, j)$. This means when computing $opt(i, j')$, we don't have to consider as many splitting points!

To minimize the runtime, we apply the idea behind divide and conquer. First, compute $opt(i, n/2)$. Then, compute $opt(i, n/4)$, knowing that it is less than or equal to $opt(i, n/2)$ and $opt(i, 3n/4)$ knowing that it is greater than or equal to $opt(i, n/2)$. By recursively keeping track of the lower and upper bounds on opt , we reach a $O(mn \log n)$ runtime. Each possible value of $opt(i, j)$ only appears in $\log n$ different nodes.

Note that it doesn't matter how "balanced" $opt(i, j)$ is. Across a fixed level, each value of k is used at most twice, and there are at most $\log n$ levels.

Generic implementation

Even though implementation varies based on problem, here's a fairly generic template. The function `compute` computes one row i of states `dp_cur`, given the previous row $i-1$ of states `dp_before`. It has to be called with `compute(0, n-1, 0, n-1)`. The function `solve` computes m rows and returns the result.

```
int m, n;
vector<long long> dp_before, dp_cur;

long long C(int i, int j);

// compute dp_cur[l], ... dp_cur[r] (inclusive)
void compute(int l, int r, int optl, int optr) {
    if (l > r)
        return;

    int mid = (l + r) >> 1;
    pair<long long, int> best = {LLONG_MAX, -1};

    for (int k = optl; k <= min(mid, optr); k++) {
        best = min(best, {(k ? dp_before[k-1] : 0) + C(k, mid), k});
    }

    dp_cur[mid] = best.first;
    int opt = best.second;

    compute(l, mid-1, optl, opt);
    compute(mid+1, r, opt, optr);
}
```

```

long long solve() {
    dp_before.assign(n, 0);
    dp_cur.assign(n, 0);

    for (int i = 0; i < n; i++)
        dp_before[i] = C(0, i);

    for (int i = 1; i < m; i++) {
        compute(0, n - 1, 0, n - 1);
        dp_before = dp_cur;
    }

    return dp_before[n - 1];
}

```

Things to look out for

The greatest difficulty with Divide and Conquer DP problems is proving the monotonicity of opt . One special case where this is true is when the cost function satisfies the quadrangle inequality, i.e., $C(a, c) + C(b, d) \leq C(a, d) + C(b, c)$ for all $a \leq b \leq c \leq d$. Many Divide and Conquer DP problems can also be solved with the Convex Hull trick or vice-versa. It is useful to know and understand both!

Practice Problems

- [AtCoder - Yakiniku Restaurants](#)
- [CodeForces - Ciel and Gondolas](#) (Be careful with I/O!)
- [CodeForces - Levels And Regions](#)
- [CodeForces - Partition Game](#)
- [CodeForces - The Bakery](#)
- [CodeForces - Yet Another Minimization Problem](#)
- [Codechef - CHEFAOR](#)
- [CodeForces - GUARDS](#) (This is the exact problem in this article.)
- [Hackerrank - Guardians of the Lunatics](#)
- [Hackerrank - Mining](#)
- [Kattis - Money \(ACM ICPC World Finals 2017\)](#)
- [SPOJ - ADAMOLD](#)
- [SPOJ - LARMY](#)
- [SPOJ - NKLEAVES](#)
- [Timus - Bicolored Horses](#)
- [USACO - Circular Barn](#)
- [UVA - Arranging Heaps](#)
- [UVA - Naming Babies](#)

References

- [Quora Answer by Michael Levin](#)
- [Video Tutorial by "Sothe" the Algorithm Wolf](#)

Contributors:

[Arnav Sastry](#) (45.97%)
 [Sabit Zahin](#) (31.45%)
 [Dustin Miao](#) (5.65%)
 [Michael Hayter](#) (4.03%)
 [Oleksandr Kulkov](#) (4.03%)
 [omkarbajaj073](#) (2.42%)
[DrDrunkenstein42](#) (1.61%)
 [18o3](#) (0.81%)
 [Alpacabla](#) (0.81%)
 [CDTheGod](#) (0.81%)
 [Jakob Kogler](#) (0.81%)
 [Tarun Khullar](#) (0.81%)
 [simonyanli](#) (0.81%)

Original

Knuth's Optimization

Knuth's optimization, also known as the Knuth-Yao Speedup, is a special case of dynamic programming on ranges, that can optimize the time complexity of solutions by a linear factor, from $O(n^3)$ for standard range DP to $O(n^2)$.

Conditions

The Speedup is applied for transitions of the form

$$dp(i, j) = \min_{i \leq k < j} [dp(i, k) + dp(k + 1, j) + C(i, j)].$$

Similar to [divide and conquer DP](#), let $opt(i, j)$ be the value of k that minimizes the expression in the transition (opt is referred to as the "optimal splitting point" further in this article). The optimization requires that the following holds:

$$opt(i, j - 1) \leq opt(i, j) \leq opt(i + 1, j).$$

We can show that it is true when the cost function C satisfies the following conditions for $a \leq b \leq c \leq d$:

1. $C(b, c) \leq C(a, d)$;
2. $C(a, c) + C(b, d) \leq C(a, d) + C(b, c)$ (the quadrangle inequality [QI]).

This result is proved further below.

Algorithm

Let's process the dp states in such a way that we calculate $dp(i, j - 1)$ and $dp(i + 1, j)$ before $dp(i, j)$, and in doing so we also calculate $opt(i, j - 1)$ and $opt(i + 1, j)$. Then for calculating $opt(i, j)$, instead of testing values of k from i to $j - 1$, we only need to test from $opt(i, j - 1)$ to $opt(i + 1, j)$. To process (i, j) pairs in this order it is sufficient to use nested for loops in which i goes from the maximum value to the minimum one and j goes from $i + 1$ to the maximum value.

Generic implementation

Though implementation varies, here's a fairly generic example. The structure of the code is almost identical to that of Range DP.

```
int solve() {
    int N;
    ... // read N and input
    int dp[N][N], opt[N][N];

    auto C = [&](int i, int j) {
        ... // Implement cost function C.
    };

    for (int i = 0; i < N; i++) {
        opt[i][i] = i;
        ... // Initialize dp[i][i] according to the problem
    }

    for (int i = N-2; i >= 0; i--) {
        for (int j = i+1; j < N; j++) {
            int mn = INT_MAX;
            int cost = C(i, j);
            for (int k = opt[i][j-1]; k <= min(j-1, opt[i+1][j]); k++) {
                if (mn >= dp[i][k] + dp[k+1][j] + cost) {
                    opt[i][j] = k;
                }
            }
        }
    }
}
```

```

        mn = dp[i][k] + dp[k+1][j] + cost;
    }
    dp[i][j] = mn;
}
return dp[0][N-1];
}

```

Complexity

A complexity of the algorithm can be estimated as the following sum:

$$\sum_{i=1}^N \sum_{j=i+1}^N [\text{opt}(i+1, j) - \text{opt}(i, j-1)] = \sum_{i=1}^N \sum_{j=i}^{N-1} [\text{opt}(i+1, j+1) - \text{opt}(i, j)].$$

As you see, most of the terms in this expression cancel each other out, except for positive terms with $j = N - 1$ and negative terms with $i = 1$. Thus, the whole sum can be estimated as

$$\sum_{k=1}^N [\text{opt}(k, N) - \text{opt}(1, k)] = O(n^2),$$

rather than $O(n^3)$ as it would be if we were using a regular range DP.

On practice

The most common application of Knuth's optimization is in Range DP, with the given transition. The only difficulty is in proving that the cost function satisfies the given conditions. The simplest case is when the cost function $C(i, j)$ is simply the sum of the elements of the subarray $S[i, i+1, \dots, j]$ for some array (depending on the question). However, they can be more complicated at times.

Note that more than the conditions on the dp transition and the cost function, the key to this optimization is the inequality on the optimum splitting point. In some problems, such as the optimal binary search tree problem (which is, incidentally, the original problem for which this optimization was developed), the transitions and cost functions will be less obvious, however, one can still prove that $\text{opt}(i, j-1) \leq \text{opt}(i, j) \leq \text{opt}(i+1, j)$, and thus, use this optimization.

Proof of correctness

To prove the correctness of this algorithm in terms of $C(i, j)$ conditions, it suffices to prove that

$$\text{opt}(i, j-1) \leq \text{opt}(i, j) \leq \text{opt}(i+1, j)$$

assuming the given conditions are satisfied.

Lemma

$\text{dp}(i, j)$ also satisfies the quadrangle inequality, given the conditions of the problem are satisfied.

 Proof


The proof for this lemma uses strong induction. It has been taken from the paper [Efficient Dynamic Programming Using Quadrangle Inequalities](#), authored by F. Frances Yao, which introduced the Knuth-Yao Speedup (this particular statement is Lemma 2.1 in the paper). The idea is to induct on the length $l = d - a$. The case where $l = 1$ is trivial. For $l > 1$ consider 2 cases:

1. $b = c$

The inequality reduces to $dp(a, b) + dp(b, d) \leq dp(a, d)$ (This assumes that $dp(i, i) = 0$ for all i , which is the case for all problems using this optimization). Let $opt(a, d) = z$.

- If $z < j$,
Note that

$$dp(a, b) \leq dp_z(a, b) = dp(a, z) + dp(z + 1, b) + C(a, b).$$

Therefore,

$$dp(a, b) + dp(b, d) \leq dp(a, z) + dp(z + 1, b) + dp(b, d) + C(a, b)$$

From the induction hypothesis, $dp(z + 1, b) + dp(b, d) \leq dp(z + 1, d)$. Also, it is given that $C(a, b) \leq C(a, d)$. Combining these 2 facts with above inequality yields the desired result.

- If $z \geq j$, the proof of this case is symmetric to the previous case.

2. $b < c$

Let $opt(b, c) = z$ and $opt(a, d) = y$.

- If $z \leq y$,

$$dp(a, c) + dp(b, d) \leq dp_z(a, c) + dp_y(b, d)$$

where

$$dp_z(a, c) + dp_y(b, d) = C(a, c) + C(b, d) + dp(a, z) + dp(z + 1, c) + dp(b, y) + dp(y + 1, d).$$

Using the QI on C and on the dp state for the indices $z + 1 \leq y + 1 \leq c \leq d$ (from the induction hypothesis) yields the desired result.

- If $z > y$, the proof of this case is symmetric to the previous case.

This completes the proof of the lemma.

Now, consider the following setup. We have 2 indices $i \leq p \leq q < j$. Set $dp_k = C(i, j) + dp(i, k) + dp(k + 1, j)$.

Suppose we show that

$$dp_p(i, j - 1) \geq dp_q(i, j - 1) \implies dp_p(i, j) \geq dp_q(i, j).$$

Setting $q = opt(i, j - 1)$, by definition, $dp_p(i, j - 1) \geq dp_q(i, j - 1)$. Therefore, applying the inequality to all $i \leq p \leq q$, we can infer that $opt(i, j)$ is at least as much as $opt(i, j - 1)$, proving the first half of the inequality.

Now, using the QI on some indices $p + 1 \leq q + 1 \leq j - 1 \leq j$, we get

$$\begin{aligned} & dp(p + 1, j - 1) + dp(q + 1, j) \leq dp(q + 1, j - 1) + dp(p + 1, j) \\ \implies & (dp(i, p) + dp(p + 1, j - 1) + C(i, j - 1)) + (dp(i, q) + dp(q + 1, j) + C(i, j)) \\ & \leq (dp(i, q) + dp(q + 1, j - 1) + C(i, j - 1)) + (dp(i, p) + dp(p + 1, j) + C(i, j)) \\ \implies & dp_p(i, j - 1) + dp_q(i, j) \leq dp_p(i, j) + dp_q(i, j - 1) \\ \implies & dp_p(i, j - 1) - dp_q(i, j - 1) \leq dp_p(i, j) - dp_q(i, j) \end{aligned}$$

Finally,

$$\begin{aligned} dp_p(i, j-1) &\geq dp_q(i, j-1) \\ \implies 0 &\leq dp_p(i, j-1) - dp_q(i, j-1) \leq dp_p(i, j) - dp_q(i, j) \\ \implies dp_p(i, j) &\geq dp_q(i, j) \end{aligned}$$

This proves the first part of the inequality, i.e., $opt(i, j-1) \leq opt(i, j)$. The second part $opt(i, j) \leq opt(i+1, j)$ can be shown with the same idea, starting with the inequality $dp(i, p) + dp(i+1, q) \leq dp(i+1, p) + dp(i, q)$.

This completes the proof.

Practice Problems

- [UVA - Cutting Sticks](#)
- [UVA - Prefix Codes](#)
- [SPOJ - Breaking String](#)
- [UVA - Optimal Binary Search Tree](#)

References

- [Geeksforgeeks Article](#)
- [Doc on DP Speedups](#)
- [Efficient Dynamic Programming Using Quadrangle Inequalities](#)

Contributors:

[Oleksandr Kulkov](#) (70.77%) [DrDrunkenstein42](#) (28.21%) [FloppaInspector](#) (0.51%) [Michael Hayter](#) (0.51%)

Translated From: e-maxx.ru

Dynamic Programming on Broken Profile. Problem "Parquet"

Common problems solved using DP on broken profile include:

- finding number of ways to fully fill an area (e.g. chessboard/grid) with some figures (e.g. dominoes)
- finding a way to fill an area with minimum number of figures
- finding a partial fill with minimum number of unfilled space (or cells, in case of grid)
- finding a partial fill with the minimum number of figures, such that no more figures can be added

Problem "Parquet"

Problem description. Given a grid of size $N \times M$. Find number of ways to fill the grid with figures of size 2×1 (no cell should be left unfilled, and figures should not overlap each other).

Let the DP state be: $dp[i, mask]$, where $i = 1, \dots, N$ and $mask = 0, \dots, 2^M - 1$.

i represents number of rows in the current grid, and $mask$ is the state of last row of current grid. If j -th bit of $mask$ is 0 then the corresponding cell is filled, otherwise it is unfilled.

Clearly, the answer to the problem will be $dp[N, 0]$.

We will be building the DP state by iterating over each $i = 1, \dots, N$ and each $mask = 0, \dots, 2^M - 1$, and for each $mask$ we will be only transitioning forward, that is, we will be *adding* figures to the current grid.

Implementation

```
int n, m;
vector < vector<long long> > dp;

void calc (int x = 0, int y = 0, int mask = 0, int next_mask = 0)
{
    if (x == n)
        return;
    if (y >= m)
        dp[x+1][next_mask] += dp[x][mask];
    else
    {
        int my_mask = 1 << y;
        if (mask & my_mask)
            calc (x, y+1, mask, next_mask);
        else
        {
            calc (x, y+1, mask, next_mask | my_mask);
            if (y+1 < m && ! (mask & my_mask) && ! (mask & (my_mask << 1)))
                calc (x, y+2, mask, next_mask);
        }
    }
}

int main()
{
    cin >> n >> m;

    dp.resize (n+1, vector<long long> (1<<m));
    dp[0][0] = 1;
    for (int x=0; x<n; ++x)
        for (int mask=0; mask<(1<<m); ++mask)
```

```
        calc (x, 0, mask, 0);  
  
        cout << dp[n][0];  
  
    }
```

Practice Problems

- [UVA 10359 - Tiling](#)
- [UVA 10918 - Tri Tiling](#)
- [SPOJ GNY07H \(Four Tiling\)](#)
- [SPOJ M5TILE \(Five Tiling\)](#)
- [SPOJ MNTILE \(MxN Tiling\)](#)
- [SPOJ DOJ1](#)
- [SPOJ DOJ2](#)
- [SPOJ BTCODE_J](#)
- [SPOJ PBOARD](#)
- [ACM HDU 4285 - Circuits](#)
- [LiveArchive 4608 - Mosaic](#)
- [Timus 1519 - Formula 1](#)
- [Codeforces Parquet](#)

References

- [Blog by EvilBunny](#)
- [TopCoder Recipe by "syg96"](#)
- [Blogpost by sk765](#)

Contributors:

[Silap Aliyev](#) (52.75%) [Aditya Dutt](#) (36.26%) [Oleksandr Kulkov](#) (6.59%) [Jakob Kogler](#) (3.3%) [Ahmad Karimi](#) (1.1%)

Translated From: e-maxx.ru

Finding the largest zero submatrix

You are given a matrix with n rows and m columns. Find the largest submatrix consisting of only zeros (a submatrix is a rectangular area of the matrix).

Algorithm

Elements of the matrix will be $a[i][j]$, where $i = 0 \dots n - 1$, $j = 0 \dots m - 1$. For simplicity, we will consider all non-zero elements equal to 1.

Step 1: Auxiliary dynamic

First, we calculate the following auxiliary matrix: $d[i][j]$, nearest row that has a 1 above $a[i][j]$. Formally speaking, $d[i][j]$ is the largest row number (from 0 to $i - 1$), in which there is a element equal to 1 in the j -th column. While iterating from top-left to bottom-right, when we stand in row i , we know the values from the previous row, so, it is enough to update just the elements with value 1. We can save the values in a simple array $d[i]$, $i = 1 \dots m - 1$, because in the further algorithm we will process the matrix one row at a time and only need the values of the current row.

```
vector<int> d(m, -1);
for (int i = 0; i < n; ++i) {
    for (int j = 0; j < m; ++j) {
        if (a[i][j] == 1) {
            d[j] = i;
        }
    }
}
```

Step 2: Problem solving

We can solve the problem in $O(nm^2)$ iterating through rows, considering every possible left and right columns for a submatrix. The bottom of the rectangle will be the current row, and using $d[i][j]$ we can find the top row. However, it is possible to go further and significantly improve the complexity of the solution.

It is clear that the desired zero submatrix is bounded on all four sides by some ones, which prevent it from increasing in size and improving the answer. Therefore, we will not miss the answer if we act as follows: for every cell j in row i (the bottom row of a potential zero submatrix) we will have $d[i][j]$ as the top row of the current zero submatrix. It now remains to determine the optimal left and right boundaries of the zero submatrix, i.e. maximally push this submatrix to the left and right of the j -th column.

What does it mean to push the maximum to the left? It means to find an index k_1 for which $d[i][k_1] > d[i][j]$, and at the same time k_1 - the closest one to the left of the index j . It is clear that then $k_1 + 1$ gives the number of the left column of the required zero submatrix. If there is no such index at all, then put $k_1 = -1$ (this means that we were able to extend the current zero submatrix to the left all the way to the border of matrix a).

Symmetrically, you can define an index k_2 for the right border: this is the closest index to the right of j such that $d[i][k_2] > d[i][j]$ (or m , if there is no such index).

So, the indices k_1 and k_2 , if we learn to search for them effectively, will give us all the necessary information about the current zero submatrix. In particular, its area will be equal to $(i - d[i][j]) * (k_2 - k_1 - 1)$.

How to look for these indexes k_1 and k_2 effectively with fixed i and j ? We can do that in $O(1)$ on average.

To achieve such complexity, you can use the stack as follows. Let's first learn how to search for an index k_1 , and save its value for each index j within the current row i in matrix $d1[i][j]$. To do this, we will look through all the columns j from left to right, and we will store in the stack only those columns that have $d[j]$ strictly greater than $d[i][j]$. It is clear that when moving from a column j to the next column, it is

necessary to update the content of the stack. When there is an inappropriate element at the top of the stack (i.e. $d[i][j] \leq d[i][j]$) pop it. It is easy to understand that it is enough to remove from the stack only from its top, and from none of its other places (because the stack will contain an increasing d sequence of columns).

The value $d1[i][j]$ for each j will be equal to the value lying at that moment on top of the stack.

The dynamics $d2[i][j]$ for finding the indices $k2$ is considered similar, only you need to view the columns from right to left.

It is clear that since there are exactly m pieces added to the stack on each line, there could not be more deletions either, the sum of complexities will be linear, so the final complexity of the algorithm is $O(nm)$.

It should also be noted that this algorithm consumes $O(m)$ memory (not counting the input data - the matrix $a[i][j]$).

Implementation

```
int zero_matrix(vector<vector<int>> a) {
    int n = a.size();
    int m = a[0].size();

    int ans = 0;
    vector<int> d(m, -1), d1(m), d2(m);
    stack<int> st;
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) {
            if (a[i][j] == 1)
                d[j] = i;
        }

        for (int j = 0; j < m; ++j) {
            while (!st.empty() && d[st.top()] <= d[j])
                st.pop();
            d1[j] = st.empty() ? -1 : st.top();
            st.push(j);
        }
        while (!st.empty())
            st.pop();

        for (int j = m - 1; j >= 0; --j) {
            while (!st.empty() && d[st.top()] <= d[j])
                st.pop();
            d2[j] = st.empty() ? m : st.top();
            st.push(j);
        }
        while (!st.empty())
            st.pop();

        for (int j = 0; j < m; ++j)
            ans = max(ans, (i - d[j]) * (d2[j] - d1[j] - 1));
    }
    return ans;
}
```

Contributors:

[Vlad Tiberiu Mihailescu](#) (93.62%) [Oleksandr Kulkov](#) (6.38%)